QCP：基于分离逻辑的 C 程序验证方法与实践

以 LiteOS 双向链表的初始化、插入与删除为例

戴子琛　陈怡蓉　崔家诚

24 级计科拔尖　Group6

2026 年 10 月 24 日

QCP: A Practical Separation Logic-based C Program Verification Tool

汇报主线

- 研究背景
- 理论基础
- 核心方法
- LiteOS 链表样例复现
- 结果分析

形式化验证

随着软件系统复杂度不断提高，传统测试难以覆盖所有可能的执行情况。尤其在操作系统内核等底层 C 程序中，复杂的指针操作与动态内存管理增加了正确性保障的难度。形式化验证通过数学逻辑描述程序行为，严格证明程序满足预期性质。

现有验证方法

- **完全自动化验证**：自动化程度高但主要针对特定安全性质，复杂功能正确性验证能力弱。
- **基于标注验证**：通过程序规范与自动推理进行验证，但依赖人工编写适当的逻辑标注。
- **交互式验证**：能够证明复杂程序性质，但需要大量人工证明工作。

核心挑战：难以同时兼顾复杂性质的表达能力、验证自动化程度与开发过程中的交互反馈。

QCP 的目标

结合基于标注的自动验证与交互式证明引入增量验证机制，兼顾表达能力、自动化与实时反馈。

- **复杂功能正确性验证（Expressiveness）**

  支持复杂的功能正确性性质，具备与交互式证明助手相当的表达和证明能力。

- **尽可能减少人工干预（Automation）**

  充分利用自动化推理，使简单程序及安全性质仅通过编写标注即可完成验证，无需额外构造人工证明。

- **开发过程中的实时反馈（Real-time Feedback）**

  在编写代码和标注的过程中即时执行符号分析，展示中间验证状态，并及时定位错误，而非依赖完整程序的事后验证。

Hoare 逻辑（Hoare Logic）

通过前置条件与后置条件描述程序应满足的正确性规范，基本形式为：

$$\{P\}\ C\ \{Q\}$$

前置条件 P：执行前调用者需保证的条件，如输入范围、内存访问权限。
后置条件 Q：执行后程序应保证的结果，如链表合法、排序结果有序。
验证目标：证明满足 P 时，程序 C 若正常终止，其结果必满足 Q。

分离逻辑（Separation Logic）

在 Hoare 逻辑基础上扩展了对堆内存的描述能力，适用于指针操作与动态数据结构的验证。

- **指向断言**（$x \mapsto v$）：表示地址 x 指向存储值 v 的内存单元。
- **分离合取**（$P * Q$）：表示 P 和 Q 分别在互不重叠的堆内存区域成立。
- **框架规则（Frame Rule）**：仅推理程序修改的局部内存，保留其余独立内存区域的性质。

核心作用：为链表、树等动态数据结构提供精确的内存建模与局部推理能力，是 QCP 验证指针操作和功能正确性的理论基础。

符号执行与验证条件生成

符号执行（Symbolic Execution）

使用符号变量代替具体输入，根据程序语句和路径约束推导程序状态。QCP 基于分离逻辑计算最强后置条件，跟踪程序执行过程中的内存状态。

验证条件（Verification Conditions, VCs）

将内存访问、逻辑断言等正确性要求转化为逻辑蕴含关系 $P \vdash Q$，即证明已知条件 P 能够推出目标性质 Q，从而将程序正确性验证转化为逻辑证明问题。

示例：链表节点访问验证

① 内存访问条件

执行 x->next 前，需要证明指针 x 不仅非空，还拥有访问 next 字段所需的内存资源：

$$x \neq \mathrm{NULL} \;\&\&\; \mathrm{listrep}(x) \vdash \exists\, v\ R,\ x \to \mathrm{next} == v \;\&\&\; R$$

其中，listrep(x) 是对链表结构的抽象描述，R 表示剩余内存资源断言。由于该谓词尚未展开，当前状态没有直接给出节点各字段的内存信息。

② 展开谓词并生成验证条件

用户添加断言以展开 listrep(x)，明确节点内部的内存结构。QCP 为该断言生成验证条件：

$$x \neq \mathrm{NULL} \;\&\&\; \mathrm{listrep}(x) \vdash \exists\, v\ n,\ x \to \mathrm{next} == n \;\&\&\; x \to \mathrm{data} == v \;\&\&\; \mathrm{listrep}(n)$$

该条件要求证明：非空链表可分解为当前节点的 next、data 字段及后续链表 listrep(n)。QCP 自动证明该条件后，即可利用展开后的内存信息继续执行指针读取操作。

自动求解与交互式证明

1. 自动证明（Automatic Verification）

QCP 使用分离逻辑蕴含求解器（Entailment Solver）自动处理生成的验证条件，内部结合两类推理机制：

- **分离逻辑规则推理**：通过谓词展开与规则应用，处理堆内存的空间约束。
- **轻量 SMT 求解器**：支持线性整数算术（LIA）与未解释函数（UF），自动处理相应的数值及等式约束。

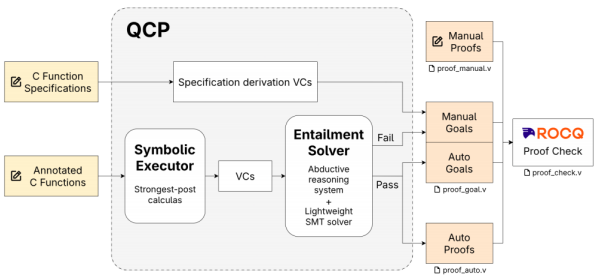
2. 交互式证明（Interactive Proving）

对于自动求解器无法完成的复杂验证条件，QCP 将剩余证明义务交给 Rocq：

- 用户借助分离逻辑引理与证明策略补充人工证明。
- Rocq 检查自动生成及人工补充的证明，确保验证条件得到正确证明。

核心思想：简单验证条件自动完成，复杂验证条件由 Rocq 补充证明，实现自动化效率与复杂性质证明能力的结合。

QCP 系统架构



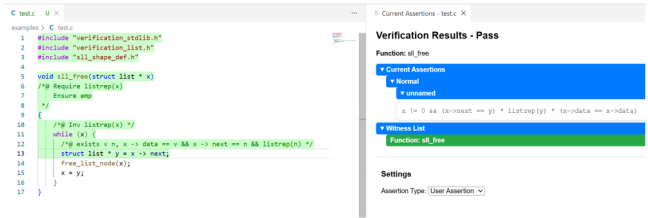
1. **程序规范输入：**接收带有逻辑标注的 C 函数及其函数规范，支持函数实现验证与规范推导验证。
2. **验证条件生成：**符号执行器计算最强后置条件，生成实现验证 VC；函数规范另行生成规范推导 VC。
3. **自动化验证：**通过分离逻辑蕴含求解器与内置轻量 SMT 求解器，自动证明能够处理的验证条件。
4. **交互式证明：**用户在 Rocq 中补充剩余目标的证明，Rocq 统一检查自动证明与人工证明。

- **自动验证与交互式证明相结合**

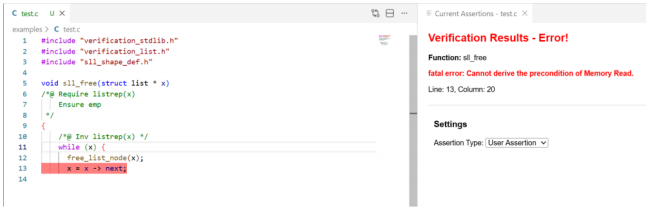
  结合基于标注的自动验证与 Rocq 交互式证明：简单验证条件由自动求解器处理，复杂验证条件允许用户补充人工证明，在保证表达能力的同时减少证明工作量。

- **增量验证与实时反馈**

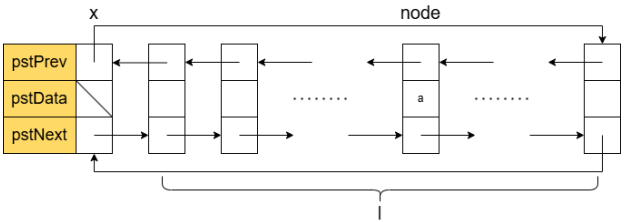
  通过 VS Code 插件及 Web 界面，将验证过程融入程序开发。支持对尚未完成的程序进行增量符号执行，实时展示中间状态、验证结果与错误位置，帮助开发者逐步完善程序与逻辑标注。



验证通过：展示当前断言与中间状态



验证失败：定位内存读取错误及其位置



链表定义

```
typedef struct LOS_DL_LIST {
    int pstData;
    struct LOS_DL_LIST *pstPrev, *pstNext;
} LOS_DL_LIST;
```

x 为哨兵，pstPrev、pstNext 分别连接前驱与后继；空表的两个指针均指向自身。

链表谓词：store_dll(x)

store_dll(x) 表示以 *x* 为哨兵的循环双向链表，概括链表的内存资源与前后指针的连接关系。论文中的这一谓词关注链表的整体结构，不区分节点存储的具体数据。在函数规范中，用它描述操作前后的链表状态：初始化建立空链表，插入和删除后仍保持合法的循环双向链表结构。

链表初始化：从字段赋值到空表规范

LOS_ListInit

```
static inline void LOS_ListInit(LOS_DL_LIST *list)
/*@ With {A} (storeA: Z -> A -> Assertion)
    Require undef_data_at(&(list -> pstPrev)) *
            undef_data_at(&(list -> pstNext))
    Ensure store_dll(storeA, list, nil)
*/
{
    list->pstNext = list;
    list->pstPrev = list;
}
```

前置条件：拥有可写字段

undef_data_at 表示拥有字段的内存资源，但不约束其旧值。分离合取 $*$ 表示两个字段占用互不重叠的内存区域。

后置条件：建立合法空表

两次赋值使 pstNext 和 pstPrev 均指向 list。

store_dll(storeA, list, nil) 表示以 list 为哨兵、节点序列为空的循环双向链表。

关键验证条件与证明

执行后的字段状态，需要能够推出空链表谓词：

$$\text{list.pstPrev} \mapsto \text{list} \;*\; \text{list.pstNext} \mapsto \text{list} \;\vdash\; \text{store_dll(storeA, list, nil)}$$

Rocq 证明：根据空链表的定义，证明两个指针均指向哨兵的状态满足后置条件，从而确认初始化正确。

链表插入：底层规范与指针更新

LOS_ListAdd

```
/*@ primitive_spec
  With (x: Z) (a: Z)
  Require list->pstNext == x && x->pstPrev == list &&
    node->pstData == a &&
    has_permission(&(node->pstPrev)) *
    has_permission(&(node->pstNext))
  Ensure x->pstPrev == node && node->pstNext == x &&
    list->pstNext == node && node->pstPrev == list &&
    node->pstData == a */;
node->pstNext = list->pstNext;
node->pstPrev = list;
list->pstNext->pstPrev = node;
list->pstNext = node;
```

前置条件：拥有相邻链接与新节点

x 为 list 的原后继，两者相互连接。新节点的数据为 a，并拥有其两个链接字段的访问权限。

后置条件：建立新的双向连接

node 被插入 list 与 x 之间，四个链接字段建立新的双向关系，节点数据 a 保持不变。

$$\text{list} \leftrightarrow \text{node} \leftrightarrow x$$

关键验证条件与证明

四次赋值后的字段状态，需要满足 list、node 与 x 之间的新连接关系，并保持节点数据不变。

Rocq 证明：检查更新后的四个链接字段与后置条件逐一对应，确认节点被正确插入。

多层规范：头插抽象与尾插复用

头插规范：head_insert_spec

```
/*@ head_insert_spec <= primitive_spec
  With (a: Z)
  Require node->pstData == a &&
    store_dll(list) *
    has_permission(&(node->pstPrev)) *
    has_permission(&(node->pstNext))
  Ensure store_dll(list) */
```

尾插实现：LOS_ListTailInsert

```
{ LOS_ListAdd(list->pstPrev, node); }
```

前置条件：合法链表与新节点

原链表结构合法，新节点数据为 a，并拥有新节点两个链接字段的访问权限。

后置条件：保持完整链表结构

头插在哨兵之后加入节点；尾插在原尾节点之后加入节点。两者完成后，链表均满足 store_dll(list)。

关键验证条件与证明

更新链接后恢复完整链表；head_insert_spec <= primitive_spec 表示由底层规范导出头插规范。

Rocq 证明：结合更新后的链接与未修改的链表部分，证明插入后仍是一条合法的循环双向链表。

链表删除：摘除节点与保留资源

LOS_ListDelete

```
static inline void LOS_ListDelete(LOS_DL_LIST *node)
{
    node->pstNext->pstPrev = node->pstPrev;
    node->pstPrev->pstNext = node->pstNext;
    node->pstNext = (LOS_DL_LIST *)0;
    node->pstPrev = (LOS_DL_LIST *)0;
}
```

前置条件：目标节点位于链表中

node 是链表中的一个普通节点，前后链接合法，并拥有修改相关字段所需的内存资源。

后置条件：恢复剩余链表

前驱与后继直接相连，剩余节点构成合法链表。被摘节点及其关联数据的内存仍保留；规范中的 undef_data_at 不约束其链接字段的具体值。

关键验证条件与证明

四次赋值后的状态，需要满足“剩余链表结构合法，且被摘节点的内存资源仍保留”的后置条件。

Rocq 证明：利用链段拆分与拼接引理，证明跳过目标节点后，前后两部分能重新组成合法链表。

验证条件的自动处理：SMT 与分离逻辑

前面的样例给出了前后置条件；QCP 通过符号执行生成验证条件，再由蕴含求解器尝试自动证明。

纯算术条件：删除时的零值范围

删除操作将两个链接字段置零，产生整数范围条件：

```
0 <= INT_MAX && INT_MIN <= 0
```

- **目标：**证明写入的常量 0 在整数表示范围内。
- **SMT 处理：**利用线性整数算术（LIA）判断该范围条件成立。
- **作用：**自动处理数值与等式约束，减少逐条编写证明的工作。

空间条件：尾插前的资源整理

调用尾插操作前，需要整理节点与链表资源：

```
N ** H |-- H ** N
```

- *N* 表示新节点资源，*H* 表示原链表资源。
- **目标：**重排资源，匹配插入函数调用的前置条件。
- **分离逻辑处理：**应用资源交换、谓词展开与匹配规则，处理内存条件。

证明后端：补充证明与统一检查

自动求解未完成的条件由用户在 Rocq 中补充证明，自动与人工证明最终统一接受检查。

初始化样例的检查模块

```
Module VC_Correctness : VC_Correct.
  Include common_strategy_proof.
  Include los_sortlink_strategy_proof.
  Include LOS_ListInit_proof_auto.
  Include LOS_ListInit_proof_manual.
End VC_Correctness.
```

四类文件的作用

- goal.v：定义验证目标与接口。
- proof_auto.v：自动目标对应的证明。
- proof_manual.v：需补充的证明。
- goal_check.v：汇集证明并检查接口覆盖。

对应的链表样例

- **初始化：**证明两个自指字段满足空链表的定义。
- **插入：**证明局部链接更新后，整体链表仍合法。
- **删除：**利用链段引理，证明剩余部分可组成合法链表。

与 OpenHarmony 链表实现对照

对照对象：作者 QCP v1.0 样例与 OpenHarmony kernel_liteos_a/kernel/include/los_list.h。

QCP 样例（函数体）

初始化：LOS_ListInit

```
list->pstNext = list;
list->pstPrev = list;
```

插入：LOS_ListAdd

```
node->pstNext = list->pstNext;
node->pstPrev = list;
list->pstNext->pstPrev = node;
list->pstNext = node;
```

删除：LOS_ListDelete

```
node->pstNext->pstPrev = node->pstPrev;
node->pstPrev->pstNext = node->pstNext;
node->pstNext = (LOS_DL_LIST *)0;
node->pstPrev = (LOS_DL_LIST *)0;
```

OpenHarmony 对应实现（函数体）

初始化：LOS_ListInit

```
list->pstNext = list;
list->pstPrev = list;
```

插入：LOS_ListAdd

```
node->pstNext = list->pstNext;
node->pstPrev = list;
list->pstNext->pstPrev = node;
list->pstNext = node;
```

删除：LOS_ListDelete

```
node->pstNext->pstPrev = node->pstPrev;
node->pstPrev->pstNext = node->pstNext;
node->pstNext = NULL;
node->pstPrev = NULL;
```

QCP 应用场景与局限性

论文实验：155 个函数，**1964/2643** 个验证条件自动完成（**74.3%**）

应用场景

1. **动态数据结构验证**

   验证链表、树、数组等数据结构的指针操作、内存安全性及结构不变量。

2. **算法正确性验证**

   应用于归并排序、KMP 字符串匹配等典型算法以及 QCP 自身部分算法实现正确性验证。

3. **操作系统内核组件验证**

   面向 LiteOS 内核中的双向链表，验证插入、删除等操作是否满足指定的内存与结构约束。

局限性

1. **C 语言特性支持不完整**

   暂不支持浮点类型、函数指针、位域和 goto 等部分 C 语言特性。

2. **自动求解能力有限**

   内置 SMT 求解器主要支持线性整数算术（LIA）与未解释函数（UF），部分复杂性质仍需借助 Rocq 人工证明。

3. **依赖人工标注与证明**

   需用户编写函数规范、循环不变量等逻辑标注，复杂验证任务仍存在一定人工证明成本。

LiteOS 链表样例复现结果

复现完成情况

- **样例覆盖：**初始化、插入、尾插与删除，共 4 个函数。
- **验证流程：**生成验证条件，复用已有证明，完成所选 Rocq 检查模块的编译。
- **源码对照：**样例与 OpenHarmony 对应函数的核心指针操作一致。

证明结果与分析

- **验证条件：**展示的 4 个函数共生成 14 项直接验证条件，结果与作者基线一致。
- **已完成证明：**14 项验证条件已有证明，并以 Qed 结束。

Thanks